

LeetCode 732: My Calendar III

Authored by: Arpit Raj, LNMIIT Jaipur | Date: August 15, 2026

Solution & Notes

This implementation uses a **Sweep-Line Algorithm** to efficiently track the maximum number of concurrent events. By recording the start of an event as a +1 and the end as a -1, we can sort these timeline markers. Iterating through the sorted markers and maintaining a running prefix sum allows us to dynamically find the peak number of overlapping calendar bookings (the ans).

```
class MyCalendarThree {
public:
    MyCalendarThree() {}

    vector<pair<int, int>> booking;
    int book(int starttime, int endtime) {

        booking.push_back({starttime, 1});
        booking.push_back({endtime, -1});

        sort(booking.begin(), booking.end());

        int ans = 0;
        int cnt = 0;

        for(auto &it : booking){
            cnt += it.second;
            ans = max(ans, cnt);
        }

        return ans;
    }
};
```

```
/**  
 * Your MyCalendarThree object will be instantiated and called  
as such:  
 * MyCalendarThree* obj = new MyCalendarThree();  
 * int param_1 = obj->book(startTime,endTime);  
 */
```